

Flight Director

Code Quality Audit Report

Project	clawdview
Audit date	2026-05-06
Languages	TypeScript, JavaScript, React
Source files	38
Machine	@andrewhopper/fd v0.1.3 — code-quality-audit.yaml
Branch	claude/flight-control-quality-audit-wKwmq

Score card

Principle	Score	Grade	Crit	Warn
1. No Defaults	5/10	C	0	6
2. Config Abstracted	4/10	D	1	2
3. Simplicity	6/10	C	1	1
4. Strongly Typed	6/10	C	1	3
5. Decoupled	8/10	B	0	1
6. DRY	5/10	C	1	2
7. Architectural	7/10	B	0	2
OVERALL	5.9/10	C	4	17

Overall: 5.9/10 (C). Solid bones — the layered architecture is clean and there are no circular imports. Headline issues are (1) absent config layer, (2) two oversized files, (3) pervasive any, and (4) duplicated CLI entry points.

How to use this report

Each finding has three approval boxes — **Approve** (do it), **Defer** (track but skip for now), **Reject** (won't do). Tick one and write a comment in the lined area. I'll process your annotations after you sync the PDF back from your reMarkable.

Findings

F-01 · WARNING

No Defaults

Magic default port and host via || fallback

clawdview-tool/src/server/server.ts:32-33

What: this.port = options.port || 3333; this.host = options.host || 'localhost'. These silently provide defaults instead of failing fast. The same magic literal 3333 reappears in server.ts:187, tunnel-service.ts:40, bin/clawdview.js:35, and bin/cview.js:111.

Suggested fix: Extract DEFAULT_PORT and DEFAULT_HOST to src/server/config.ts. Use ?? not ||. Centralize so the literal 3333 lives in exactly one place.

Effort: Quick win (~20 min)

■ Approve

■ Defer

■ Reject

Comments / next step:

F-02 · WARNING

No Defaults

Implicit fallback to process.cwd()

clawdview-tool/src/server/server.ts:35

What: options.watchDir || process.cwd(). The fallback is invisible to callers; a missing watchDir silently watches whatever the cwd happens to be.

Suggested fix: Make watchDir required, or log explicitly when defaulting. Move to config module.

Effort: Quick win (~10 min)

■ Approve

■ Defer

■ Reject

Comments / next step:

Hardcoded CORS origins duplicated

clawdview-tool/src/server/server.ts:41, 162

What: Two locations build the same CORS origin array literal [`http://localhost:${port}``, `http://127.0.0.1:${port}``. Magic strings, repeated.

Suggested fix: Extract buildCorsOrigins(port) helper or move to a config object.

Effort: Quick win (~15 min)

☐ Approve☐ Defer☐ Reject

Comments / next step:

Tuning constants embedded in service module

clawdview-tool/src/server/services/python-executor.ts:5-9

What: RATE_LIMIT_COUNT=5, RATE_LIMIT_WINDOW=60000, TIMEOUT=30000, MAX_CODE_SIZE=50000 are named constants (good) but live in the service file itself. They control operational policy, not service mechanics.

Suggested fix: Move to a config module so an operator can tune them without touching code.

Effort: Medium (~1 hr)

☐ Approve☐ Defer☐ Reject

Comments / next step:

No dedicated config module; no startup validation`clawdview-tool/src/server/`

What: `process.env.PORT` is read inline in `server.ts:187`. There is no `config.ts`, no schema (zod/valibot/pydantic-style), no startup validation. Adding new env vars will scatter env reads across the codebase.

Suggested fix: Create `src/server/config.ts` that reads + validates all env vars at startup with a zod schema and exports a typed config object.

Effort: Medium (~2 hrs)

☐ Approve☐ Defer☐ Reject

Comments / next step:

F-06 · WARNING

Per-service constants instead of layered config`clawdview-tool/src/server/services/python-executor.ts`

What: Operational limits (timeout, rate limit, max size) are baked into the python executor. Cannot vary by environment (dev/test/prod).

Suggested fix: Source these from the central config module created in F-05.

Effort: Medium (~30 min, after F-05)

☐ Approve☐ Defer☐ Reject

Comments / next step:

File length 660 lines (threshold 500)`clawdview-tool/src/client/components/FileTree.tsx`

What: FileTree.tsx mixes view-mode logic, search, collapse state, multi-dir add/remove UI, file-type grouping, and drag/keyboard interaction in one component.

Suggested fix: Decompose: extract , , , and a useFileTreeState() hook. Target <300 lines per file.

Effort: Structural (1-2 days)

☐ Approve☐ Defer☐ Reject

Comments / next step:

File length 449 lines (threshold 300)`clawdview-tool/src/client/App.tsx`

What: App.tsx owns routing-ish state, theme, preferences glue, file fetch, error handling, and the highlight.js bridge.

Suggested fix: Extract data-fetch logic into a useCurrentFile() hook; move the hljs bridge to a dedicated module.

Effort: Medium (~3 hrs)

☐ Approve☐ Defer☐ Reject

Comments / next step:

Pervasive use of `any`

12 occurrences across 8 files

What: Notable cases: App.tsx:25 `declare const hljs: any`; App.tsx:85 `(data as any).error`; tunnel-service.ts:90,111 `let localtunnel: any` for dynamic imports; six `catch (e: any)` blocks (format-routes, execute-routes, file-routes, App.tsx, JsonRenderer, ReactRenderer).

Suggested fix: (a) Use `@types/highlight.js` instead of `declare const hljs: any`. (b) Replace `catch (e: any)` with `catch (e: unknown)` plus a narrow helper `toErrorMessage(e: unknown): string`. (c) Type localtunnel/ngrok via their `@types` packages or hand-rolled minimal interfaces.

Effort: Medium (~3 hrs)

☐ Approve☐ Defer☐ Reject

Comments / next step:

F-10 · WARNING***Lazy type assertion to bypass the type system***

cawdview-tool/src/client/App.tsx:85

What: `(data as any).error || 'Failed to load file'` — silences the compiler but loses runtime safety.

Suggested fix: Define a typed `FileLoadResponse` interface and validate the payload shape.

Effort: Quick win (~20 min)

☐ Approve☐ Defer☐ Reject

Comments / next step:

noUnusedLocals / noUnusedParameters disabled`clawdview-tool/tsconfig.app.json`

What: Strict mode is on, but ``noUnusedLocals`` and ``noUnusedParameters`` are explicitly set to false in `tsconfig.app.json` — dead code accumulates silently.

Suggested fix: Flip both to true and clean up the resulting warnings.

Effort: Quick win (~30 min)

☐ Approve☐ Defer☐ Reject

Comments / next step:

Layered architecture is intact`clawdview-tool/src/server/`

What: No circular imports detected. `routes/` → `services/` → `shared/` flows correctly. `tunnel-service.ts` uses `dynamic require()` for optional deps — acceptable.

Suggested fix: No action required. Document the layering in `CLAUDE.md` to lock it in.

Effort: Quick win (~15 min, optional)

☐ Approve☐ Defer☐ Reject

Comments / next step:

Two near-duplicate CLI entry points

clawdview-tool/bin/clawdview.js (210 LOC) + bin/cview.js (134 LOC)

What: Both files implement the same `start` command with overlapping option parsing, tunnel setup, and host/port logic. The diff is mostly cosmetic.

Suggested fix: Consolidate into a single CLI module that both bin shims delegate to. Each shim becomes ~3 lines.

Effort: Medium (~2 hrs)

☐ Approve☐ Defer☐ Reject

Comments / next step:

CORS origin construction duplicated

clawdview-tool/src/server/server.ts:41 + 162

What: Same array literal built twice. Already covered structurally by F-03 fix.

Suggested fix: Resolved by F-03.

Effort: Quick win (folded into F-03)

☐ Approve☐ Defer☐ Reject

Comments / next step:

Repeated `catch (e: any)` + `e.message` pattern

format-routes.ts:25, execute-routes.ts:28, file-routes.ts:52,65, App.tsx:145,
JsonRenderer.tsx:11, ReactRenderer.tsx:42

What: Six+ catch blocks that all do the same coercion to a string error message.

Suggested fix: Extract `toErrorMessage(e: unknown): string` into shared/errors.ts (also resolves part of F-09).

Effort: Quick win (~20 min)

☐ Approve☐ Defer☐ Reject

Comments / next step:

Architectural pattern is not declared

Repository root (README.md, CLAUDE.md)

What: The codebase follows an implicit layered/MVC-ish pattern (routes → services → shared types) and React component-based UI, but no document spells it out. New contributors have to reverse-engineer it.

Suggested fix: Add an ARCHITECTURE.md (or section in CLAUDE.md) stating: server is layered (routes/services/shared); client is component-based; cross-cutting concerns (logging, errors) live in shared/.

Effort: Quick win (~45 min)

☐ Approve☐ Defer☐ Reject

Comments / next step:

Naming and boundaries are consistent`clawdview-tool/src/server/`

What: Services suffixed `*Service`, route factories prefixed ``create*Routes``, shared types in `shared/`.
Boundaries are clean.

Suggested fix: No action.

Effort: —

☒ Approve	☒ Defer	☒ Reject
---	---	--

Comments / next step:

Sign-off

Once you've gone through the findings, sign here so I know which version of the report I'm acting on. I'll then bundle every Approved item into a single follow-up branch.

Reviewer name: _____

Date: _____

Signature: _____

Aggregate decision (optional):

☐ Approve all quick wins	☐ Approve all critical only	☐ Hold — discuss first
---	--	---